

Pendekatan Greedy Set Cover untuk Optimasi Penempatan Access Point Wi-Fi pada Denah Ruangan

Muhammad Haris Putra Sulastianto^{1,2}

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung, Jl. Ganesha 10 Bandung 40132, Indonesia

¹13524053@std.stei.itb.ac.id

²mharisputras.work@gmail.com

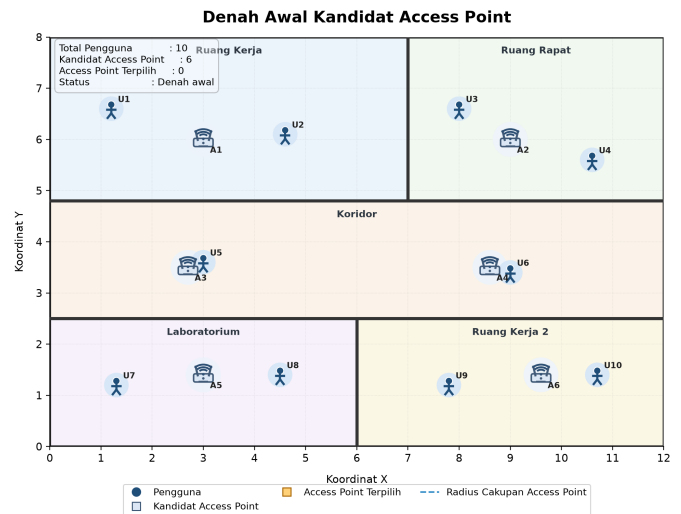
Abstrak—Penempatan access point yang kurang tepat dapat menyebabkan adanya area tanpa jangkauan jaringan, sedangkan penggunaan access point dalam jumlah berlebihan dapat meningkatkan biaya pemasangan dan pengelolaan jaringan. Makalah ini membahas penerapan algoritma Greedy Set Cover untuk menentukan lokasi penempatan access point Wi-Fi dalam jumlah yang relatif sedikit sehingga seluruh titik pengguna dapat tercakup. Denah dimodelkan sebagai grid dua dimensi yang memuat titik pengguna dan sejumlah kandidat lokasi pemasangan access point. Setiap kandidat direpresentasikan sebagai himpunan titik pengguna yang berada dalam radius cakupannya. Pada setiap iterasi, algoritma memilih kandidat access point yang mampu mencakup titik pengguna yang belum terlayani dalam jumlah terbesar. Proses tersebut dilakukan hingga seluruh titik pengguna tercakup atau tidak terdapat kandidat yang dapat memberikan cakupan tambahan. Implementasi diuji menggunakan skenario dasar dan skenario pembandingan untuk melihat hasil greedy serta perbedaannya terhadap solusi optimal yang diperoleh melalui Brute Force. Kinerja algoritma dianalisis berdasarkan jumlah access point terpilih, persentase cakupan, jumlah iterasi, dan waktu eksekusi. Hasil pengujian menunjukkan bahwa Greedy Set Cover dapat menghasilkan solusi yang sama dengan solusi optimal pada skenario tertentu, tetapi dapat berbeda pada skenario lain karena keputusan greedy hanya didasarkan pada keuntungan lokal setiap iterasi.

Kata Kunci—access point, algoritma greedy, Brute Force, jaringan Wi-Fi, optimasi penempatan, Set Cover Problem

I. PENDAHULUAN

Jaringan Wi-Fi mengacu pada keluarga standar IEEE 802.11 yang mengatur komunikasi jaringan lokal nirkabel pada lapisan MAC dan PHY [1]. Dalam penerapannya, penempatan *access point* memengaruhi cakupan jaringan Wi-Fi pada suatu ruangan. Penempatan yang kurang tepat dapat meninggalkan area tanpa jangkauan, sedangkan pemasangan terlalu banyak perangkat meningkatkan biaya. Oleh karena itu, diperlukan metode untuk memilih lokasi pemasangan yang dapat mencakup seluruh pengguna dengan jumlah *access point* yang relatif sedikit.

Gambar 1 menunjukkan contoh denah yang memuat sejumlah pengguna dan kandidat lokasi pemasangan *access point*. Setiap kandidat memiliki radius cakupan tertentu dan dapat melayani himpunan pengguna yang berbeda.



Gambar 1. Denah awal pengguna dan kandidat lokasi *access point*.

Permasalahan tersebut dimodelkan sebagai *Set Cover Problem*. Seluruh pengguna menjadi himpunan semesta, sedangkan setiap kandidat *access point* merepresentasikan himpunan pengguna yang berada dalam radius cakupannya. Solusi yang dicari adalah sekumpulan kandidat yang cakupannya meliputi seluruh pengguna.

Makalah ini menggunakan algoritma Greedy Set Cover. Pada setiap iterasi, algoritma memilih kandidat yang mencakup pengguna belum terlayani dalam jumlah terbesar. Proses diulang hingga seluruh pengguna tercakup atau tidak terdapat kandidat yang memberikan cakupan tambahan.

Implementasi dilakukan pada denah dua dimensi dan diuji menggunakan skenario dasar serta skenario pembandingan. Kinerja algoritma dianalisis berdasarkan jumlah *access point* terpilih, persentase cakupan, jumlah iterasi, waktu eksekusi, serta perbandingan terhadap solusi optimal dari Brute Force. Model yang digunakan belum memperhitungkan dinding, interferensi, kapasitas perangkat, dan pelemahan sinyal.

II. DASAR TEORI

A. Model Cakupan Access Point

Denah direpresentasikan sebagai bidang dua dimensi. Setiap pengguna dan kandidat lokasi *access point* memiliki koordinat (x, y) . Jarak antara kandidat a_i dan pengguna u_j dihitung menggunakan jarak Euclidean,

$$d(a_i, u_j) = \sqrt{(x_{a_i} - x_{u_j})^2 + (y_{a_i} - y_{u_j})^2}. \quad (1)$$

Pengguna u_j dianggap tercakup oleh kandidat a_i apabila

$$d(a_i, u_j) \leq r, \quad (2)$$

dengan r menyatakan radius cakupan. Model ini mengasumsikan bahwa seluruh *access point* memiliki radius yang sama dan tidak memperhitungkan penghalang maupun pelemahan sinyal.

B. Set Cover Problem

Set Cover Problem adalah persoalan memilih sejumlah himpunan bagian sehingga gabungannya mencakup seluruh anggota himpunan semesta [2], [3].

Misalkan himpunan seluruh pengguna dinyatakan sebagai

$$U = \{u_1, u_2, \dots, u_m\}, \quad (3)$$

dan himpunan seluruh kandidat lokasi *access point* dinyatakan sebagai

$$A = \{a_1, a_2, \dots, a_n\}. \quad (4)$$

Untuk setiap kandidat a_i , dibentuk himpunan cakupan

$$S_i = \{u_j \in U \mid d(a_i, u_j) \leq r\}. \quad (5)$$

Tujuannya adalah memilih himpunan kandidat $A' \subseteq A$ dengan ukuran minimum sehingga

$$\bigcup_{a_i \in A'} S_i = U. \quad (6)$$

Versi keputusan dari Set Cover termasuk persoalan NP-complete, sedangkan versi optimasinya termasuk NP-hard [4]. Oleh karena itu, pencarian solusi optimal dengan memeriksa seluruh kombinasi kandidat memerlukan waktu yang meningkat secara eksponensial terhadap jumlah kandidat [2].

C. Greedy Set Cover

Greedy Set Cover membangun solusi secara bertahap dengan memilih kandidat yang memberikan cakupan tambahan terbesar [5], [6].

Misalkan U' adalah himpunan pengguna yang belum tercakup. Keuntungan kandidat a_i didefinisikan sebagai

$$\text{gain}(a_i) = |S_i \cap U'|. \quad (7)$$

Jika A' adalah himpunan kandidat yang telah dipilih, kandidat berikutnya ditentukan dengan

$$a^* = \arg \max_{a_i \in A \setminus A'} |S_i \cap U'|. \quad (8)$$

Setelah a^* dipilih, himpunan pengguna yang belum tercakup diperbarui menjadi

$$U' \leftarrow U' \setminus S_{a^*}. \quad (9)$$

Proses dihentikan ketika U' kosong atau seluruh kandidat yang tersisa memiliki keuntungan nol.

Algoritma greedy tidak selalu menghasilkan solusi optimal. Untuk m anggota pada himpunan semesta, ukuran solusi yang dihasilkan memiliki faktor aproksimasi paling besar

$$H_m = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{m} \leq 1 + \ln m \quad (10)$$

terhadap solusi optimal [2], [5].

D. Kompleksitas Algoritma

Misalkan n adalah jumlah kandidat dan m adalah jumlah pengguna. Pembentukan himpunan cakupan untuk seluruh kandidat memerlukan waktu

$$O(nm). \quad (11)$$

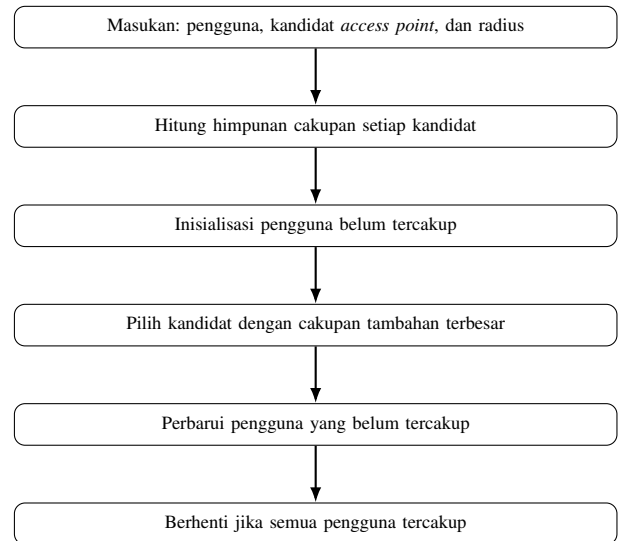
Pada setiap iterasi, implementasi sederhana memeriksa paling banyak n kandidat dan menghitung cakupan terhadap paling banyak m pengguna. Karena setiap kandidat terpilih harus mencakup setidaknya satu pengguna baru, jumlah iterasi paling banyak $\min(n, m)$. Dengan demikian, kompleksitas waktu terburuk algoritma adalah

$$O(nm \min(n, m)). \quad (12)$$

Penyimpanan seluruh himpunan cakupan memerlukan ruang paling banyak

$$O(nm). \quad (13)$$

III. IMPLEMENTASI



Gambar 2. Alur penerapan Greedy Set Cover pada penempatan *access point*.

Program diimplementasikan menggunakan Python. Data skenario disimpan dalam berkas JSON, sedangkan visualisasi denah dibuat menggunakan pustaka Matplotlib. Program menghasilkan denah awal, hasil pemilihan *access point*, log setiap iterasi, dan ringkasan hasil pengujian.

A. Representasi Data Masukan

Masukan program terdiri atas ukuran denah, radius cakupan, posisi pengguna, dan kandidat lokasi *access point*. Setiap pengguna dan kandidat direpresentasikan oleh sebuah identitas serta koordinat pada bidang dua dimensi.

Contoh representasi pengguna dan kandidat adalah sebagai berikut.

```
"users": {
  "U1": [1.2, 6.6],
  "U2": [4.6, 6.1]
},
"candidates": {
  "A1": [3.0, 6.0],
  "A2": [9.0, 6.0]
}
```

Data ruangan dan dinding juga disimpan di dalam skenario, tetapi hanya digunakan untuk visualisasi. Perhitungan cakupan hanya menggunakan koordinat pengguna, koordinat kandidat, dan radius *access point*.

B. Pembentukan Himpunan Cakupan

Program menghitung jarak setiap kandidat *access point* terhadap seluruh pengguna. Jika jaraknya tidak melebihi radius, pengguna dimasukkan ke dalam himpunan cakupan kandidat tersebut.

Hasil perhitungan disimpan dalam bentuk pemetaan

$$a_i \mapsto S_i, \quad (14)$$

dengan S_i merupakan himpunan pengguna yang dapat dicakup oleh kandidat a_i . Pembentukan seluruh himpunan cakupan dilakukan sebelum algoritma greedy dijalankan.

C. Penerapan Greedy Set Cover

Program menyimpan himpunan pengguna yang belum tercakup dalam variabel *uncovered*. Pada setiap iterasi, program menghitung jumlah pengguna baru yang dapat dicakup oleh setiap kandidat yang belum dipilih.

Kandidat dengan nilai *gain* terbesar kemudian dimasukkan ke dalam solusi. Jika terdapat beberapa kandidat dengan nilai yang sama, program memilih kandidat dengan identitas terkecil secara leksikografis agar hasil eksekusi konsisten.

Langkah algoritma yang digunakan adalah sebagai berikut.

```
1:  $U' \leftarrow U$ 
2:  $A' \leftarrow \emptyset$ 
3: while  $U' \neq \emptyset$  do
4:   Pilih  $a^* \in A \setminus A'$  yang memaksimalkan  $|S_{a^*} \cap U'|$ 
5:   if  $|S_{a^*} \cap U'| = 0$  then
6:     break
7:   end if
```

```
8:    $A' \leftarrow A' \cup \{a^*\}$ 
9:    $U' \leftarrow U' \setminus S_{a^*}$ 
10: end while
11: return  $A'$ 
```

Algoritma berhenti ketika seluruh pengguna telah tercakup atau tidak terdapat kandidat yang mampu mencakup pengguna tambahan. Kondisi kedua menunjukkan bahwa sebagian pengguna tidak dapat dijangkau oleh kandidat yang tersedia.

D. Penerapan Brute Force Set Cover

Selain Greedy Set Cover, program juga menyediakan metode Brute Force Set Cover sebagai pembanding. Berbeda dengan greedy yang memilih kandidat secara bertahap berdasarkan cakupan tambahan terbesar, Brute Force mencoba seluruh kombinasi kandidat dari ukuran terkecil.

Misalkan terdapat n kandidat *access point*. Program membangkitkan seluruh kombinasi kandidat berukuran 1, kemudian 2, dan seterusnya hingga n . Untuk setiap kombinasi, program menghitung gabungan cakupan dari seluruh kandidat di dalam kombinasi tersebut. Jika gabungan cakupan sudah mencakup seluruh pengguna, kombinasi tersebut dikembalikan sebagai solusi optimal berdasarkan jumlah kandidat minimum.

Langkah Brute Force Set Cover yang digunakan adalah sebagai berikut.

```
1: Urutkan seluruh kandidat dalam  $A$ 
2: for  $k = 1$  hingga  $|A|$  do
3:   for setiap kombinasi  $C \subseteq A$  dengan ukuran  $k$  do
4:     Hitung  $S_C = \bigcup_{a_i \in C} S_i$ 
5:     if  $S_C = U$  then
6:       return  $C$ 
7:     end if
8:   end for
9: end for
10: return solusi parsial terbaik jika tidak ada kombinasi yang mencakup seluruh pengguna
```

Metode ini menjamin solusi dengan jumlah *access point* minimum karena kombinasi diperiksa dari ukuran terkecil. Namun, jumlah kombinasi yang harus diperiksa dapat mencapai 2^n , sehingga Brute Force hanya praktis untuk skenario kecil. Pada makalah ini, Brute Force digunakan bukan sebagai metode utama, melainkan sebagai pembanding untuk menilai kualitas solusi greedy.

E. Visualisasi Hasil

Program menghasilkan gambar denah awal serta gambar hasil pemilihan kandidat. Pada mode greedy, program menyimpan visualisasi hasil Greedy Set Cover. Jika opsi pembanding dijalankan, program juga menyimpan visualisasi hasil Brute Force Set Cover.

Pengguna yang telah tercakup dan pengguna yang belum tercakup ditampilkan dengan simbol yang berbeda. Visualisasi juga memuat nama ruangan, legenda, jumlah pengguna, jumlah kandidat, jumlah *access point* terpilih, dan persentase cakupan. Dengan visualisasi tersebut, perbedaan hasil antara Greedy Set Cover dan Brute Force dapat diamati secara langsung.

Tabel I
RINGKASAN DATA UJI

Parameter	Skenario Dasar	Skenario 2
Ukuran denah	12 × 8	6 × 4
Jumlah pengguna	10	6
Jumlah kandidat	6	3
Radius cakupan	3 satuan	2 satuan
Model jarak	Euclidean	Euclidean
Pengaruh dinding	Tidak dihitung	Tidak dihitung

F. Alur Program

Secara keseluruhan, program dijalankan melalui tahapan berikut.

- 1) Membaca dan memvalidasi berkas skenario.
- 2) Mengubah koordinat pengguna dan kandidat ke dalam struktur data program.
- 3) Menghitung himpunan cakupan setiap kandidat.
- 4) Menjalankan algoritma Greedy Set Cover.
- 5) Jika opsi pembandingan aktif, menjalankan Brute Force Set Cover.
- 6) Mengukur waktu eksekusi algoritma.
- 7) Menampilkan log iterasi, ringkasan hasil, dan perbandingan metode.
- 8) Mengekspor denah awal, hasil greedy, dan jika opsi pembandingan aktif, hasil brute force dalam format PNG.

Metrik yang dicatat meliputi jumlah *access point* terpilih, jumlah pengguna tercakup, persentase cakupan, jumlah iterasi, waktu eksekusi, dan jumlah kombinasi yang diperiksa oleh Brute Force.

IV. HASIL DAN PEMBAHASAN

A. Data Uji

Pengujian dilakukan pada dua skenario. Skenario pertama adalah skenario dasar yang digunakan untuk melihat proses pemilihan *access point* pada denah ruangan. Skenario kedua adalah skenario pembandingan yang dirancang untuk menunjukkan bahwa Greedy Set Cover tidak selalu menghasilkan solusi optimal global.

Pada kedua skenario, cakupan pengguna dihitung menggunakan jarak Euclidean. Dinding dan nama ruangan hanya digunakan untuk visualisasi, sehingga tidak memengaruhi perhitungan cakupan.

Selain algoritma Greedy Set Cover, program juga dikembangkan dengan pilihan algoritma Brute Force. Brute Force mencoba kombinasi kandidat dari ukuran terkecil hingga ditemukan kombinasi yang mencakup seluruh pengguna. Dengan cara tersebut, Brute Force menjamin jumlah *access point* minimum, tetapi memiliki kompleksitas eksponensial terhadap jumlah kandidat.

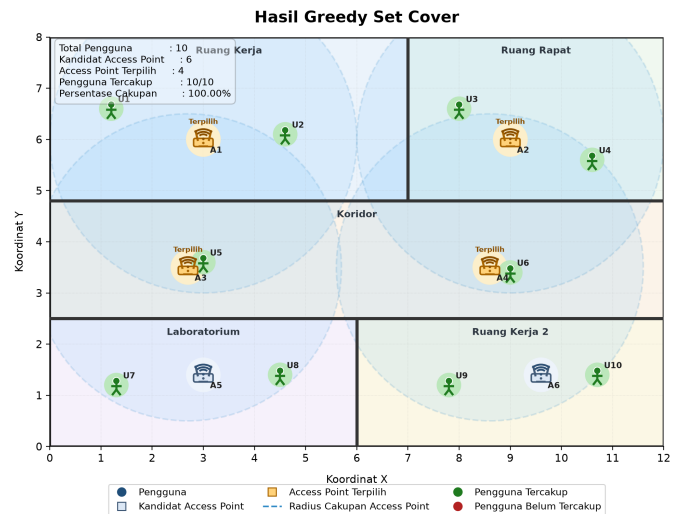
B. Skenario Dasar

Pada skenario dasar, terdapat 10 pengguna dan 6 kandidat lokasi *access point*. Berdasarkan hasil eksekusi program, Greedy Set Cover memilih 4 kandidat, yaitu A4, A1, A3, dan A2. Keempat kandidat tersebut mencakup seluruh pengguna.

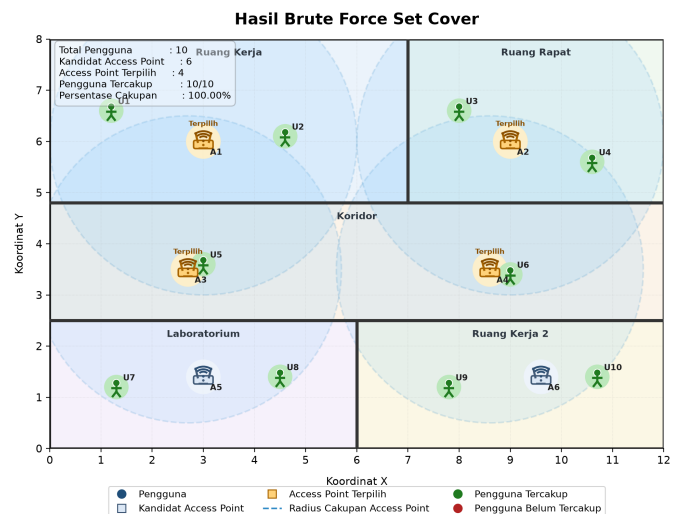
Tabel II
HASIL ITERASI GREEDY PADA SKENARIO DASAR

Iterasi	Terpilih	Pengguna Baru Tercakup	Sisa
1	A4	U4, U6, U9, U10	6
2	A1	U1, U2, U5	3
3	A3	U7, U8	1
4	A2	U3	0

Visualisasi hasil pemilihan *access point* pada skenario dasar ditunjukkan pada Gambar 3. Lingkaran cakupan menunjukkan area yang dijangkau oleh kandidat terpilih. Hasil pemilihan menggunakan Brute Force pada skenario dasar ditunjukkan pada Gambar 4.



Gambar 3. Hasil Greedy Set Cover pada skenario dasar.



Gambar 4. Hasil Brute Force Set Cover pada skenario dasar.

Pada skenario ini, Brute Force juga menemukan bahwa jumlah minimum *access point* yang diperlukan adalah 4.

Tabel III

PERBANDINGAN GREEDY DAN BRUTE FORCE PADA SKENARIO DASAR

Metode	Terpilih	Jumlah	Cakupan
Greedy Set Cover	A4, A1, A3, A2	4	100%
Brute Force	A1, A2, A3, A4	4	100%

Dengan demikian, hasil greedy sama dengan solusi optimal untuk skenario dasar.

C. Skenario Greedy Tidak Optimal

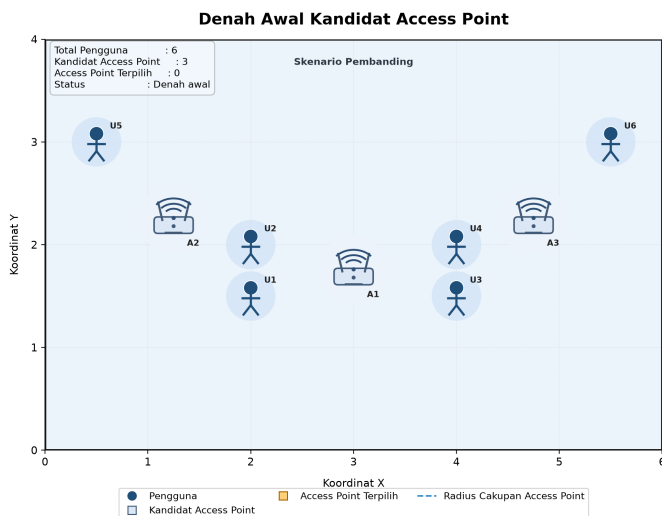
Skenario kedua digunakan untuk menunjukkan keterbatasan algoritma greedy. Skenario ini terdiri atas 6 pengguna dan 3 kandidat lokasi *access point*. Himpunan cakupan setiap kandidat ditunjukkan pada Tabel IV.

Tabel IV

HIMPUNAN CAKUPAN PADA SKENARIO GREEDY TIDAK OPTIMAL

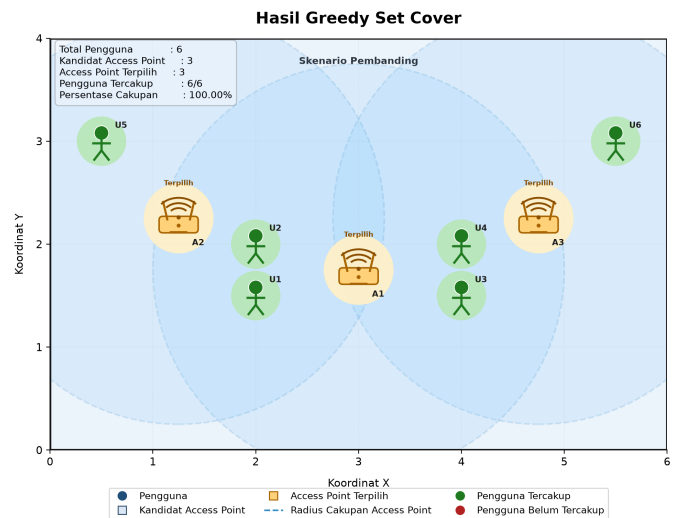
Kandidat	Pengguna Tercakup	Jumlah
A1	U1, U2, U3, U4	4
A2	U1, U2, U5	3
A3	U3, U4, U6	3

Denah awal pada skenario kedua ditunjukkan pada Gambar 5. Gambar tersebut memperlihatkan posisi 6 pengguna dan 3 kandidat lokasi *access point* sebelum algoritma dijalankan.

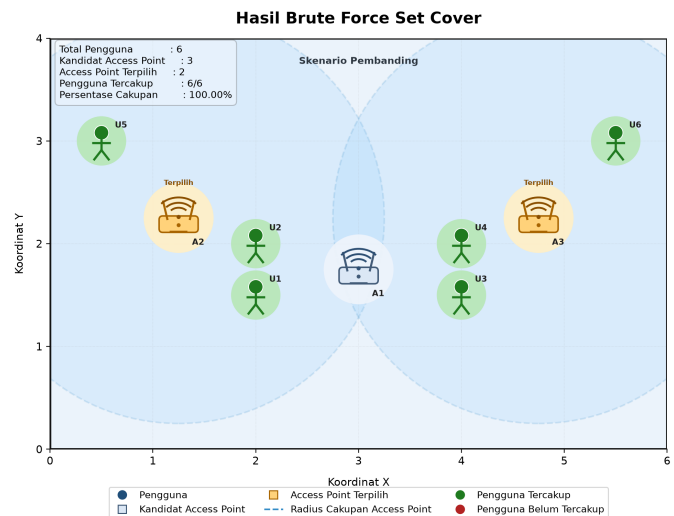


Gambar 5. Denah awal pada skenario greedy tidak optimal.

Visualisasi hasil greedy dan brute force pada skenario kedua ditunjukkan pada Gambar 6 dan Gambar 7.



Gambar 6. Hasil Greedy Set Cover pada skenario greedy tidak optimal.



Gambar 7. Hasil Brute Force Set Cover pada skenario greedy tidak optimal.

Pada skenario ini, Greedy Set Cover memilih A1 terlebih dahulu karena A1 memiliki cakupan awal terbesar, yaitu empat pengguna. Setelah A1 dipilih, pengguna yang belum tercapuk adalah U5 dan U6. Untuk mencakup kedua pengguna tersebut, algoritma kemudian memilih A2 dan A3. Dengan demikian, solusi greedy membutuhkan 3 *access point*.

Namun, solusi optimal adalah memilih A2 dan A3. Gabungan kedua kandidat tersebut sudah mencakup seluruh pengguna,

$$A2 \cup A3 = \{U1, U2, U3, U4, U5, U6\}. \quad (15)$$

Dengan demikian, Brute Force hanya membutuhkan 2 *access point*. Hasil ini menunjukkan bahwa kandidat dengan cakupan awal terbesar belum tentu menghasilkan solusi akhir dengan jumlah kandidat minimum. Pada kasus ini, keputusan lokal greedy untuk memilih A1 terlihat paling baik pada

Tabel V
PERBANDINGAN HASIL GREEDY SET COVER DAN BRUTE FORCE

Skenario	Metode	Jumlah	Cakupan
Dasar	Greedy	4	100%
Dasar	Brute Force	4	100%
Tidak optimal	Greedy	3	100%
Tidak optimal	Brute Force	2	100%

iterasi pertama, tetapi menyebabkan jumlah *access point* akhir menjadi lebih banyak dibandingkan solusi optimal.

D. Perbandingan Greedy dan Brute Force

Perbandingan hasil Greedy Set Cover dan Brute Force ditunjukkan pada Tabel V. Pada skenario dasar, greedy menghasilkan jumlah kandidat yang sama dengan Brute Force. Pada skenario kedua, greedy membutuhkan lebih banyak kandidat dibandingkan Brute Force.

Perbedaan tersebut terjadi karena greedy mengambil keputusan berdasarkan keuntungan lokal terbesar pada setiap iterasi. Sebaliknya, Brute Force memeriksa kombinasi kandidat secara menyeluruh, sehingga dapat menemukan solusi dengan jumlah kandidat minimum.

Akan tetapi, Brute Force tidak efisien untuk jumlah kandidat yang besar. Untuk n kandidat, terdapat 2^n kemungkinan subset yang dapat diperiksa. Oleh karena itu, Brute Force cocok digunakan sebagai pembanding pada skenario kecil, sedangkan Greedy Set Cover lebih praktis untuk memperoleh solusi pendekatan secara cepat.

E. Analisis Hasil

Pada skenario dasar, Greedy Set Cover menghasilkan solusi yang sama dengan Brute Force, yaitu 4 *access point* untuk mencapai cakupan 100%. Hal ini menunjukkan bahwa pada susunan pengguna dan kandidat tertentu, pilihan lokal greedy dapat bertepatan dengan solusi optimal.

Namun, skenario kedua menunjukkan bahwa greedy tidak selalu optimal. Kandidat A1 dipilih lebih dahulu karena memiliki cakupan awal terbesar. Keputusan tersebut membuat algoritma masih membutuhkan dua kandidat tambahan untuk mencakup pengguna yang tersisa. Sebaliknya, Brute Force menemukan bahwa kombinasi A2 dan A3 sudah cukup untuk mencakup seluruh pengguna.

Hasil tersebut memperlihatkan karakter utama pendekatan greedy. Greedy lebih efisien karena tidak memeriksa seluruh kombinasi kandidat, tetapi hasilnya bergantung pada keputusan lokal di setiap iterasi. Brute Force menjamin solusi optimal, tetapi jumlah kombinasi yang harus diperiksa meningkat secara eksponensial.

Dengan demikian, Greedy Set Cover lebih tepat digunakan untuk memperoleh solusi cepat dan cukup baik, sedangkan Brute Force digunakan sebagai pembanding pada ukuran data kecil untuk memverifikasi kualitas hasil greedy.

F. Keterbatasan Metode

Model yang digunakan masih menyederhanakan kondisi jaringan Wi-Fi. Dinding hanya ditampilkan sebagai bagian

dari denah dan belum memengaruhi cakupan sinyal. Selain itu, seluruh *access point* dianggap memiliki radius yang sama, tanpa memperhitungkan interferensi, kapasitas maksimum pengguna, kanal frekuensi, atau pelemahan sinyal.

Dengan batasan tersebut, hasil pengujian tidak dimaksudkan sebagai rancangan jaringan Wi-Fi nyata, melainkan sebagai simulasi penerapan Greedy Set Cover dan Brute Force pada masalah pemilihan lokasi *access point*.

V. KESIMPULAN

Algoritma Greedy Set Cover dapat digunakan untuk menentukan lokasi pemasangan *access point* berdasarkan model cakupan pada denah dua dimensi. Pada model ini, setiap kandidat *access point* direpresentasikan sebagai himpunan pengguna yang berada dalam radius cakupannya, sedangkan solusi yang dicari adalah kumpulan kandidat yang mencakup seluruh pengguna.

Pada skenario dasar berukuran 12×8 satuan dengan 10 pengguna, 6 kandidat lokasi, dan radius cakupan 3 satuan, Greedy Set Cover memilih 4 kandidat *access point*, yaitu A4, A1, A3, dan A2. Hasil tersebut mencakup seluruh pengguna dan memiliki jumlah kandidat yang sama dengan solusi Brute Force. Dengan demikian, pada skenario dasar, hasil greedy bertepatan dengan solusi optimal.

Namun, skenario pembanding menunjukkan bahwa Greedy Set Cover tidak selalu menghasilkan solusi optimal global. Pada skenario tersebut, greedy memilih A1 terlebih dahulu karena memiliki cakupan awal terbesar, sehingga membutuhkan 3 *access point*. Sebaliknya, Brute Force menemukan bahwa kombinasi A2 dan A3 sudah cukup untuk mencakup seluruh pengguna, sehingga hanya diperlukan 2 *access point*.

Berdasarkan hasil tersebut, Greedy Set Cover lebih tepat digunakan sebagai pendekatan yang efisien untuk memperoleh solusi cepat, sedangkan Brute Force digunakan sebagai pembanding pada skenario kecil karena mampu menjamin solusi optimal tetapi memiliki kompleksitas eksponensial. Pengembangan selanjutnya dapat dilakukan dengan memperhitungkan pengaruh dinding, pelemahan sinyal, interferensi, kapasitas perangkat, serta model propagasi sinyal yang lebih realistis.

PRANALA

Github: <https://github.com/mhps-null/greedy-wifi-placement>

Video: <https://youtu.be/ykBvvTNlcf>

PENUTUP

Penulis mengucapkan terima kasih kepada Tuhan Yang Maha Esa, Institut Teknologi Bandung, Program Studi Teknik Informatika, dan dosen mata kuliah IF2211 Strategi Algoritma, Bapak Rinaldi Munir, atas dukungan dalam pelaksanaan tugas makalah ini.

REFERENSI

- [1] *IEEE Standard for Information Technology–Telecommunications and Information Exchange between Systems Local and Metropolitan Area Networks–Specific Requirements Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*, IEEE Std. IEEE Std 802.11-2020, 2021, diakses 19 Juni 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/9363693>
- [2] V. V. Vazirani, *Approximation Algorithms*. Berlin, Heidelberg: Springer, 2001.
- [3] T. Stern, “Set cover problem,” [Online]. Available: <https://math.mit.edu/~goemans/18434S06/setcover-tamara.pdf>, 2006, diakses 18 Juni 2026.
- [4] R. M. Karp, “Reducibility among combinatorial problems,” in *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher, Eds. New York: Plenum Press, 1972, pp. 85–103.
- [5] V. Chvátal, “A greedy heuristic for the set-covering problem,” *Mathematics of Operations Research*, vol. 4, no. 3, pp. 233–235, Aug. 1979.
- [6] GeeksforGeeks, “Greedy approximate algorithm for set cover problem,” [Online]. Available: <https://www.geeksforgeeks.org/dsa/greedy-approximate-algorithm-for-set-cover-problem/>, 2025, diakses 18 Juni 2026.

PERNYATAAN

Dengan ini saya menyatakan bahwa makalah yang saya tulis ini adalah tulisan saya sendiri, bukan saduran, atau terjemahan dari makalah orang lain, dan bukan plagiasi.

Bandung, 19 Juni 2026



Muhammad Haris Putra Sulastianto